Design Report — CMPT 353 Project

Channel-Based Programming Q&A Tool

---

1. Architecture & Technology Choices

Next.js 15 (App Router)

The project uses a single Next.js application that serves both the UI and the API. The App Router was chosen because it collocates UI pages (`app/**/page.tsx`) and API handlers (`app/api/**/route.ts`) in the same project with no separate server process required — matching the patterns taught in the Week 11 lab. Server-rendered pages are not used; all data is fetched client-side via `fetch()` to keep the API layer testable independently.

MySQL 8

MySQL was chosen because it is the database used throughout the CMPT 353 labs and assignments, the team already had `mysql2/promise` experience, and it provides strong ACID guarantees well-suited to relational data (users → channels → posts → replies → votes). The schema is created automatically via `CREATE TABLE IF NOT EXISTS` on first container start, and seed data is inserted if the `users` table is empty. No manual migration step is needed.

Docker Compose

Two services are defined: `app` (Next.js on port 3000) and `mysql` (MySQL 8 on internal port 3306, mapped to host 3307). A healthcheck on the MySQL service ensures the app container only starts after the database is accepting connections.

---

2. API Endpoints Overview

Method	Endpoint	Auth	Purpose
GET	`/api/health`	—	DB connection check + seed counts
GET/POST	`/api/channels`	POST: user	List / create channels
DELETE	`/api/channels/:id`	admin	Remove a channel
GET/POST	`/api/channels/:id/posts`	POST: user	List / create posts
GET/DELETE	`/api/posts/:id`	DELETE: admin	Fetch / delete a post
GET/POST	`/api/posts/:id/replies`	POST: user	List / add replies

Method	Endpoint	Auth	Purpose
DELETE	`/api/replies/:id`	admin	Remove a reply
GET/POST /DELETE	`/api/votes`	write: user	Get counts, cast, change, remove vote
POST	`/api/upload`	user	Upload PNG/JPEG/WebP screenshot (≤ 5 MB)
GET	`/api/files/:id`	—	Serve uploaded file via controlled route
GET	`/api/attachments`	—	Fetch attachment metadata by target
GET	`/api/search`	—	5 query types + pagination (see Part 4)
POST	`/api/auth/register`	—	Create account (bcrypt hash)
POST	`/api/auth/login`	—	Verify credentials, set httpOnly JWT cookie
POST	`/api/auth/logout`	—	Clear JWT cookie
GET	`/api/auth/me`	—	Return current user from cookie
GET/DELETE	`/api/users` / `/api/users/:id`	admin	List / remove users
GET	`/api/admin/posts`	admin	All posts with author + channel
GET	`/api/admin/replies`	admin	All replies with author + post

All write operations (create channel/post/reply, vote, upload) return `401` if the JWT cookie is absent or invalid. Admin operations additionally return `403` if the user's role is not `admin`.

---

3. Screenshot (File Upload) Storage Approach

Uploaded images follow a three-step pipeline:

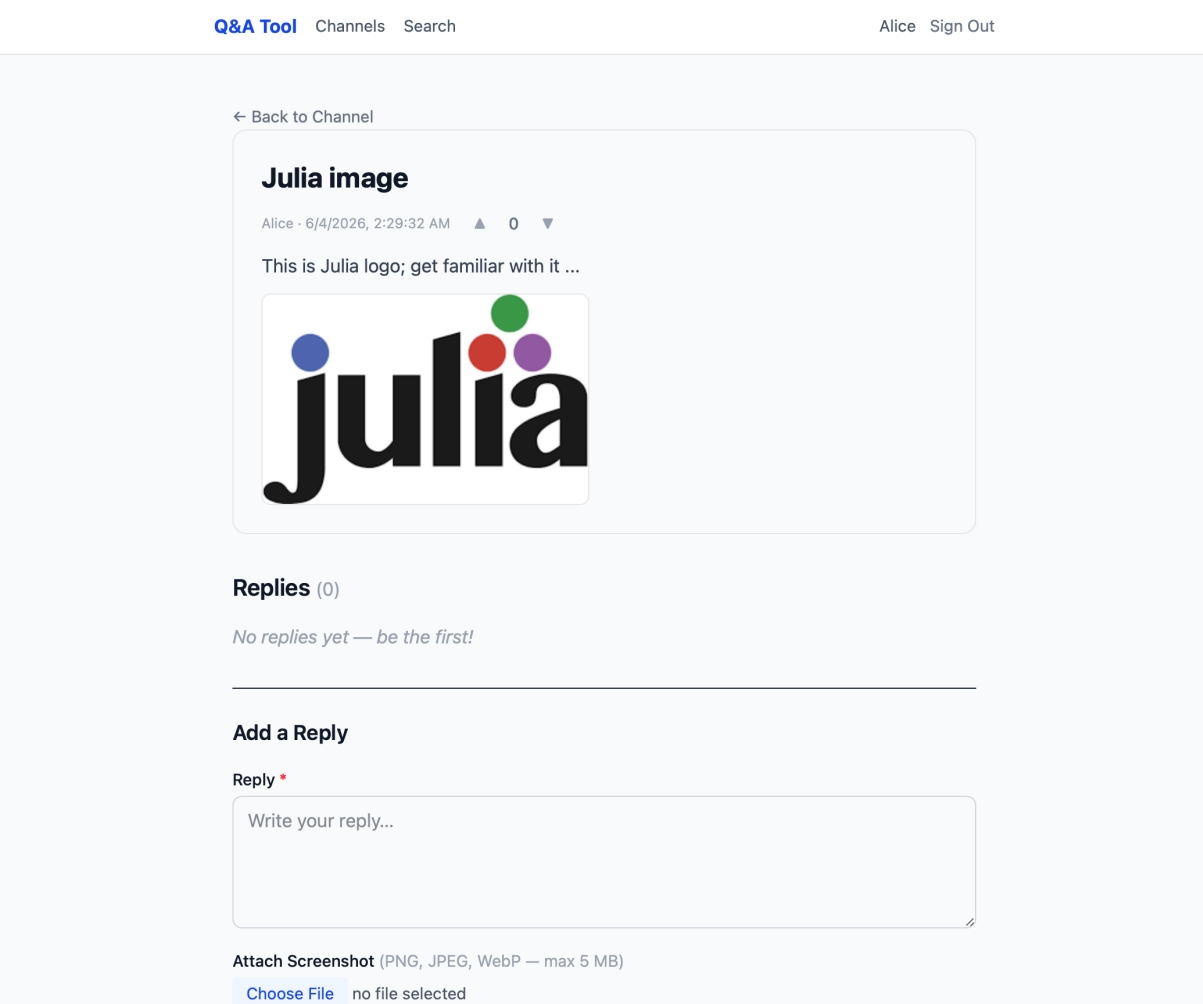
Step 1 — Validation (in `/api/upload`) Both the MIME type (`file.type`) and the file extension are checked against an allowlist (`image/png`, `image/jpeg`, `image/webp` / `.png`, `.jpg`, `.jpeg`, `.webp`). File size is capped at 5 MB. Requests from unauthenticated users are rejected before any file is read.

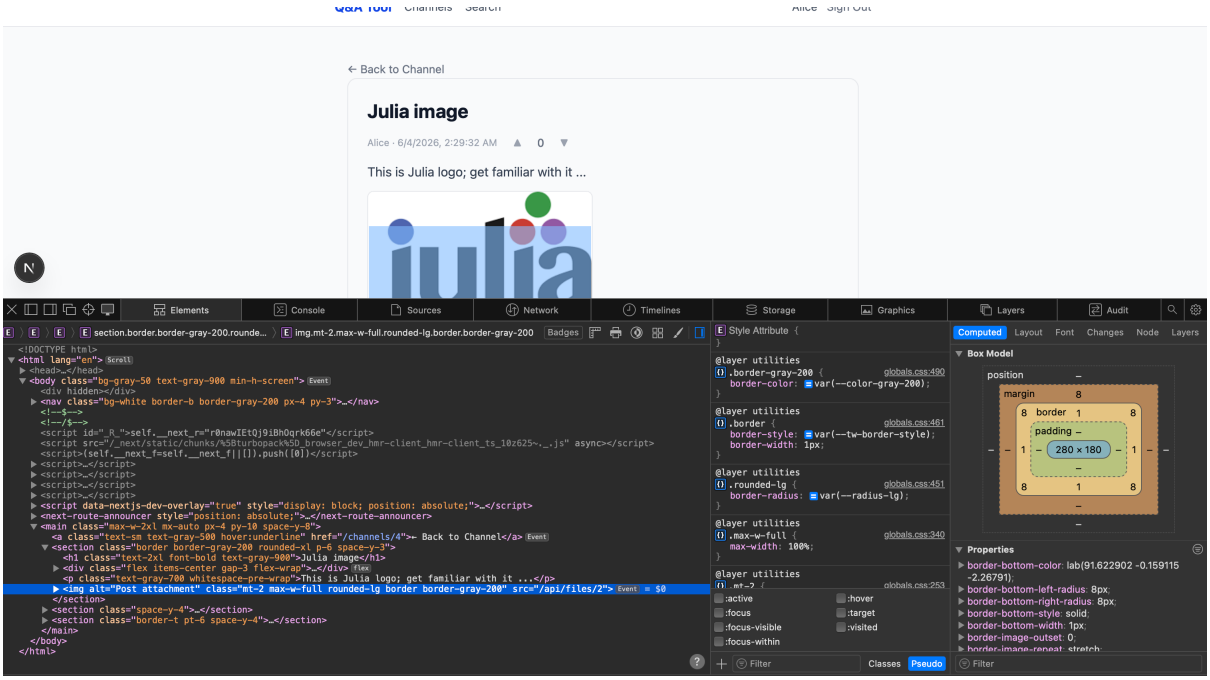
Step 2 — Storage on disk The file bytes are written to `public/uploads/<uuid>.<ext>` inside the container. A UUID filename prevents enumeration and path collisions. The `public/uploads/` directory is created automatically if it does not exist.

Step 3 — Metadata in the database A row is inserted into the `attachments` table recording `target_type` (post or reply), `target_id`, `mime_type`, `size_bytes`, and `store_path` (`/uploads/<uuid>.<ext>`). This decouples the logical reference (attachment ID) from the physical filename.

Serving files — `/api/files/:id` Files are **not** served directly from `public/`. Instead, the controlled route looks up the attachment ID in the database, resolves the disk path, reads the bytes, and returns them with the correct `Content-Type` header. This ensures invalid or deleted IDs return a clean 404 rather than exposing the filesystem.

> **Screenshot to include here:** See the uploaded image displayed on a post page — the `<img>` tag's `src` points to `/api/files/:id`, not to a raw `/uploads/` path.






```
● (base) monik@Mac CMPT_353 % cd CMPT353_Project
● (base) monik@Mac CMPT353_Project % ls
  app                     lib                     package-lock.json       README.md
  components              middleware.ts           package.json            rules.txt
  DESIGN_REPORT.md        next-env.d.ts           postcss.config.mjs      tailwind.config.ts
  docker-compose.yml      next.config.ts          Project_1.pdf           tsconfig.json
  Dockerfile              node_modules            public                  tsconfig.tsbuildinfo
● (base) monik@Mac CMPT353_Project % docker exec qa-app ls public/uploads/
  709ea088-c4b9-4e52-bac7-2b4575640a37.png
○ (base) monik@Mac CMPT353_Project %
```

```
● (base) monik@Mac CMPT353_Project % docker exec -it qa-mysql mysql -u root -prootpassword qa_db -e "SELECT * FROM attac
  hments;"
  mysql: [Warning] Using a password on the command line interface can be insecure.
  +----+-------------+-----------+-----------+------------+-----------------------------------------------------+---------
  -----------+
  | id | target_type | target_id | mime_type | size_bytes | store_path                                          | created_
  at         |
  +----+-------------+-----------+-----------+------------+-----------------------------------------------------+---------
  -----------+
  |  1 | post        |         1 | image/png |      12345 | /uploads/seed-welcome.png                           | 2026-04-
  02 01:20:23 |
  |  2 | post        |         5 | image/png |       3395 | /uploads/709ea088-c4b9-4e52-bac7-2b4575640a37.png | 2026-04-
  06 08:29:32 |
  +----+-------------+-----------+-----------+------------+-----------------------------------------------------+---------
  -----------+
○ (base) monik@Mac CMPT353_Project %
```

---

4. Key Packages

Package	Version	Why
`next`	15	Full-stack framework — UI routes + API route handlers in one project
`react` / `react-dom`	19	Component model; hooks (`useState`, `useEffect`) for client-side state
`mysql2`	^3	Async MySQL driver with Promise API (`pool.execute`); used in all

Package	Version	Why
		CMPT 353 labs
`bcrypt`	`^5`	Industry-standard password hashing with per-password salts; salt rounds = 10
`jsonwebtoken`	`^9`	Signs and verifies JWT tokens; stored as `httpOnly; SameSite=Lax` cookies to prevent XSS and basic CSRF
`tailwindcss`	`^4`	Utility-first CSS; no custom stylesheet needed for layout, spacing, or colour
No multer	—	Next.js App Router's native `req.formData()` handles `multipart/form-data` uploads without an extra middleware library

---

Word count: ~480 words · Fits on one printed page at normal margins.